

Projet programmation Algorithmique avancée

Étude et Implémentation des Structures de Tries :

Patricia-Tries et Tries Hybrides

17 Décembre, 2024

PRÉSENTÉ PAR

AHMED Youra

BENAISSA Meriem

PRÉSENTÉ À

Mme C. Truchet

Mr A.Ginitrini

Mr M. Naima

Sommaire exécutif

01 Objectifs

02 Patricia Trie

03 Insertion

04 Hybride Trie

05 Fonctions avancées

06 Fonction Complexes

07 Étude expérimentale

Objectifs

- **Représenter un dictionnaire de mots à l'aide de structures de données avancées.**
- **Comparer deux types de tries :**
 - Patricia-Tries
 - Tries Hybrides
- **Implémenter des primitives, des fonctions avancées et complexes.**
- **Réaliser une étude expérimentale sur leurs performances.**



Patricia-Trie

Définition de la structure

- Un label : préfixe partagé.
- is_end_of_word : indiquant si le nœud correspond à la fin d'un mot valide.
- Children : Une liste d'enfants stockée sous forme de map, où chaque caractère correspond à un sous-arbre.

Choix du caractère de séparation: Pip (|).

Ex : car|cat|cart|dog|bat

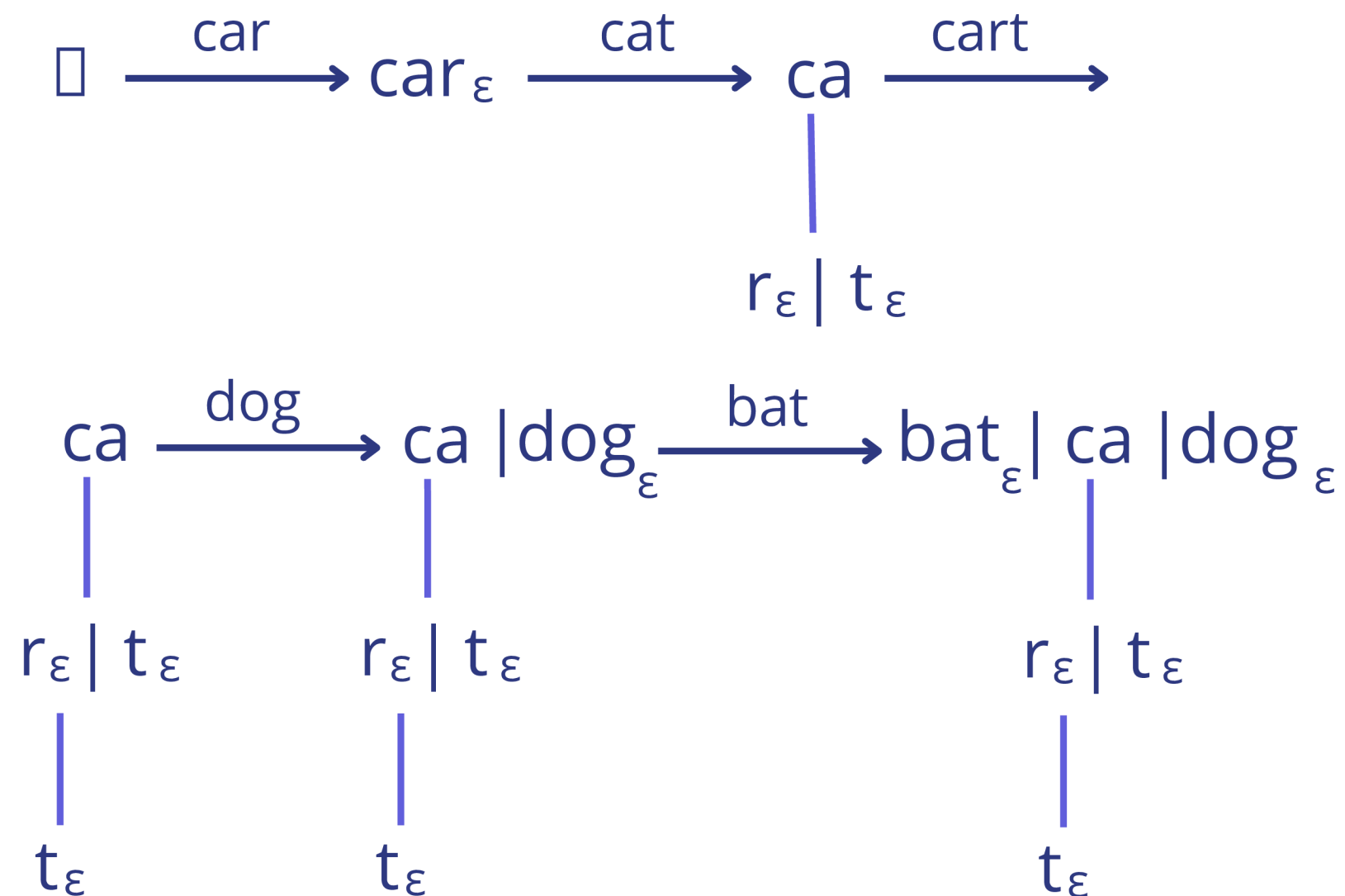
Les primitives

- **getPrefixeCommun(String s1, String s2).**
- **arrangement(PatriciaTrieNode parent, PatriciaTrieNode child, String prefixeCommun, String suffix).**
- **inserer(String mot)**
- **insertMotduFichier(String filename).**
- **insertMotsDuRepertoire(String repertoirePath)**
- **arbreToJson().**
- **saveToFile(String filename).**
- **jsonToArbre(String filename).**
- **getChildren()**
- **setChildren(Map<Character, PatriciaTrieNode> children)**
- **setIs_end_of_word(boolean is_end_of_word)**
- **isIs_end_of_word()**

insertion:

- `insérer(String mot)` : Ajoute un mot dans l'arbre.
- `insertMotduFichier(String filename)` : Insère les mots d'un fichier dans l'arbre.
- `insertMotsDuRepertoire(String repertoirePath)` : Insère les mots de plusieurs fichiers d'un répertoire.

Exemple de construction



Représentation JSON :

```
1. {
2.   "label" : "",
3.   "is_end_of_word" : false,
4.   "children" : {
5.     "b" : {
6.       "label" : "bat",
7.       "is_end_of_word" : true,
8.       "children" : { }
9.     },
10.    "c" : {
11.      "label" : "ca",
12.      "is_end_of_word" : false,
13.      "children" : {
14.        "r" : {
15.          "label" : "r",
16.          "is_end_of_word" : true,
17.          "children" : {
18.            "t" : {
19.              "label" : "t",
20.              "is_end_of_word" : true,
21.              "children" : { }
22.            }
23.          }
24.        },
25.        "t" : {
26.          "label" : "t",
27.          "is_end_of_word" : true,
28.          "children" : { }
29.        }
30.      }
31.    },
32.    "d" : {
33.      "label" : "dog",
34.      "is_end_of_word" : true,
35.      "children" : { }
36.    }
37.  }
38. }
```



Trie Hybride

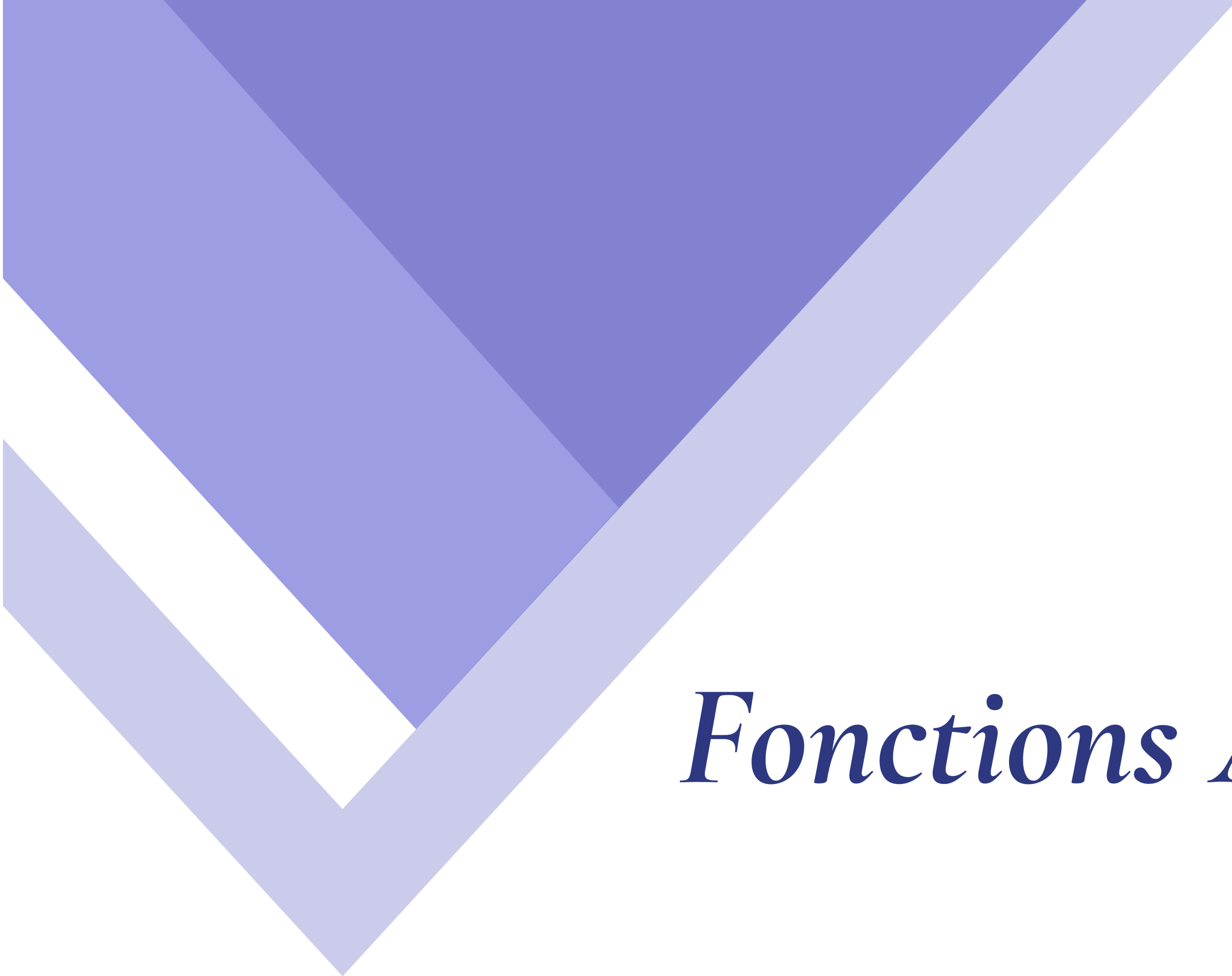
Définition de la structure

- caractere : la valeur du nœud.
- left : Un pointeur vers le fils gauche.
- middle :Un pointeur vers le fils milieu.
- right : Un pointeur vers le fils droit.
- isEndOfWord : Un indicateur de fin de mot.

Les primitives

- **insert(TrieHybridesNode trieH, String mot).**
- **insertMotduFichier(String filename) .**
- **insertMotsDuRepertoire(String repertoirePath)**
- **arbreToJson().**
- **saveToFile(String filename).**
- **jsonToArbre(String filename).**
- **rotationDroite(TrieHybridesNode arbre).**
- **rotationGauche(TrieHybridesNode arbre)**





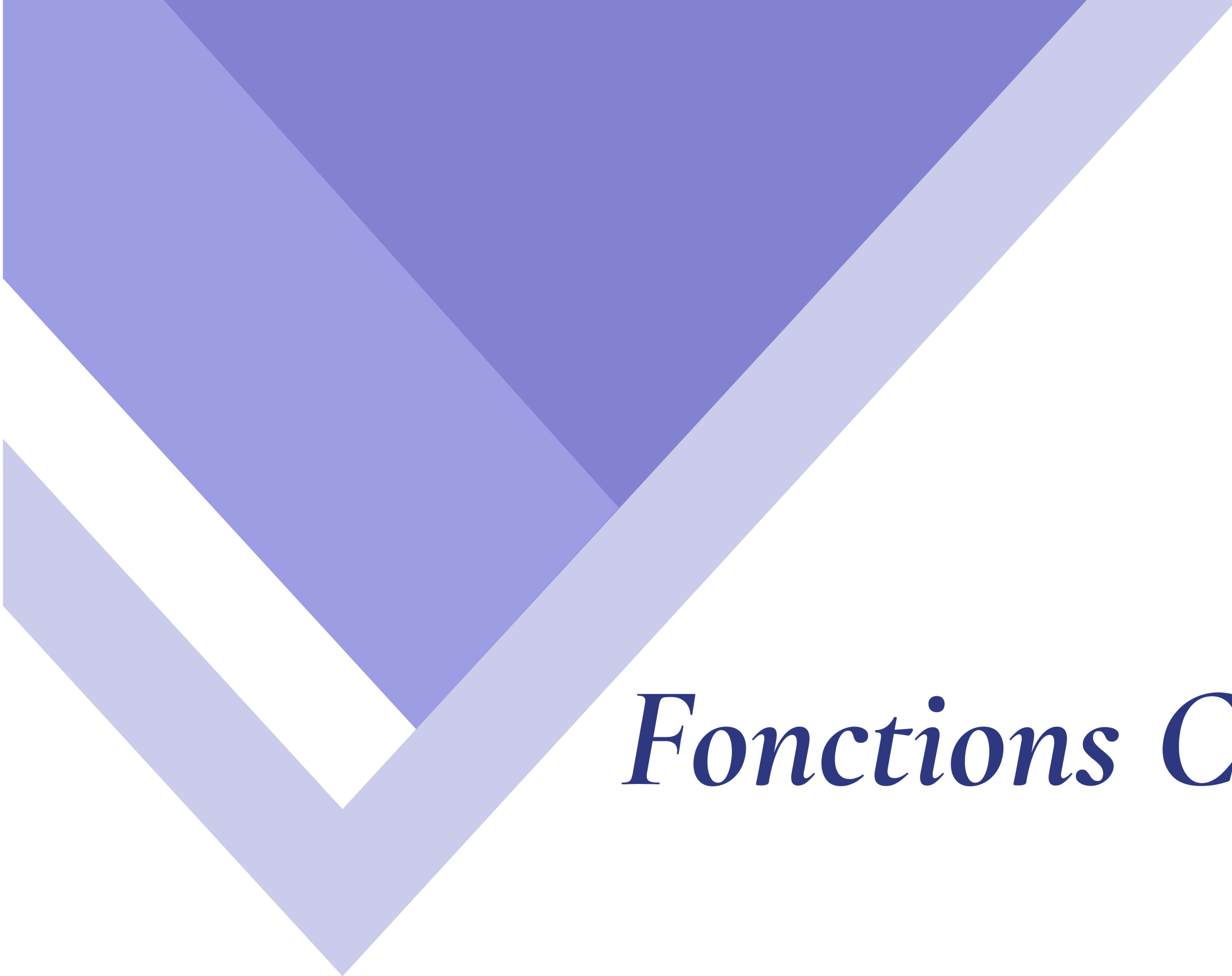
Fonctions Avancées

Complexité pour la patricia

- **Recherche(arbre, mot) booléen :**
 $O(n)$ où n est la longueur du mot, car on parcourt chaque caractère dans le pire des cas.
- **ComptageMots(arbre) entier :**
 $O(n)$ où n est le nombre total de nœuds dans l'arbre.
- **ListeMots(arbre) liste[mots] :**
 $O(n * k * \log(k))$ où n est le nombre de nœuds dans l'arbre, et k est le nombre moyen d'enfants par nœud.
- **ComptageNil(arbre) entier**
 $O(n)$ où n est le nombre total de nœuds dans l'arbre.
- **Hauteur(arbre) entier**
 $O(n)$ où n est le nombre total de nœuds dans l'arbre.
- **ProfondeurMoyenne(arbre) entier**
 $O(n)$ où n est le nombre total de nœuds dans l'arbre.
- **Prefixe(arbre, mot) entier**
 $O(p+k)$ où p est la longueur du préfixe et k le nombre de mots descendants.
- **Suppression(arbre, mot) arbre**
 $O(n)$ où n est la longueur du mot.

Complexité pour l'hybride

- **Recherche(arbre, mot) booléen :**
 $O(n + h)$ où n est la longueur du mot et h est la hauteur de l'arbre.
- **ComptageMots(arbre) entier :**
 $O(n)$ où n est le nombre total de nœuds dans l'arbre..
- **ListeMots(arbre) liste[mots] :**
 $O(n)$, où n est le nombre total de nœuds dans le trie hybride.
- **ComptageNil(arbre) entier**
 $O(n)$, où n est le nombre total de nœuds dans le trie hybride.
- **Hauteur(arbre) entier**
 $O(n)$ où n est le nombre total de nœuds dans l'arbre hybride.
- **ProfondeurMoyenne(arbre) entier**
 $O(n)$ où n est le nombre total de nœuds dans l'arbre hybride.
- **Prefixe(arbre, mot) entier**
 $O(m + k)$, où m est la longueur du préfixe et k est le nombre de nœuds du sous-trie correspondant.
- **Suppression(arbre, mot) arbre**
 $O(h+m)$, où h est la hauteur du trie et m est la longueur du mot à supprimer.



Fonctions Complexes

Fonction complexe

fusionner(PatriciaTrieNode trie1, PatriciaTrieNode trie2)

1. **Initialisation** : Créez un nouveau nœud racine pour l'arbre fusionné.
2. **Fusion des nœuds** : Parcourez les nœuds des deux arbres.
 - a. **Nœuds existants** : Si un nœud avec la même clé existe :
 - i. Comparez leurs préfixes.
 - ii. En cas de préfixe commun, fusionnez récursivement leurs sous-arbres.
 - iii. Sinon, créez un nœud commun et réorganisez leurs enfants.
 - b. **Nouveaux nœuds** : Si le nœud n'existe pas, ajoutez-le directement.
3. **Itération** : Répétez pour tous les nœuds des deux arbres.

Complexité : $O(n_1+n_2)$ sont les nombres de nœuds dans les deux arbres.

La fonction equilibrer(TrieHybridesNode arbre)

1. Calculer les hauteurs des sous-arbres gauche et droit..
2. Si la différence de hauteur est supérieure ou égale à 2 :
 - a. Si le sous-arbre gauche est plus profond :
 - i. Vérifier si une rotation gauche doit être effectuée sur le sous-arbre gauche
 - ii. Effectuer une rotation droite sur le nœud courant.
 - b. Si le sous-arbre droit est plus profond :
 - i. Vérifier si une rotation droite doit être effectuée sur le sous-arbre droit.
 - ii. Effectuer une rotation gauche sur le nœud courant.
3. Retourner le nœud rééquilibré.

Complexité : $O(h)$ où h est la hauteur du trie hybride.

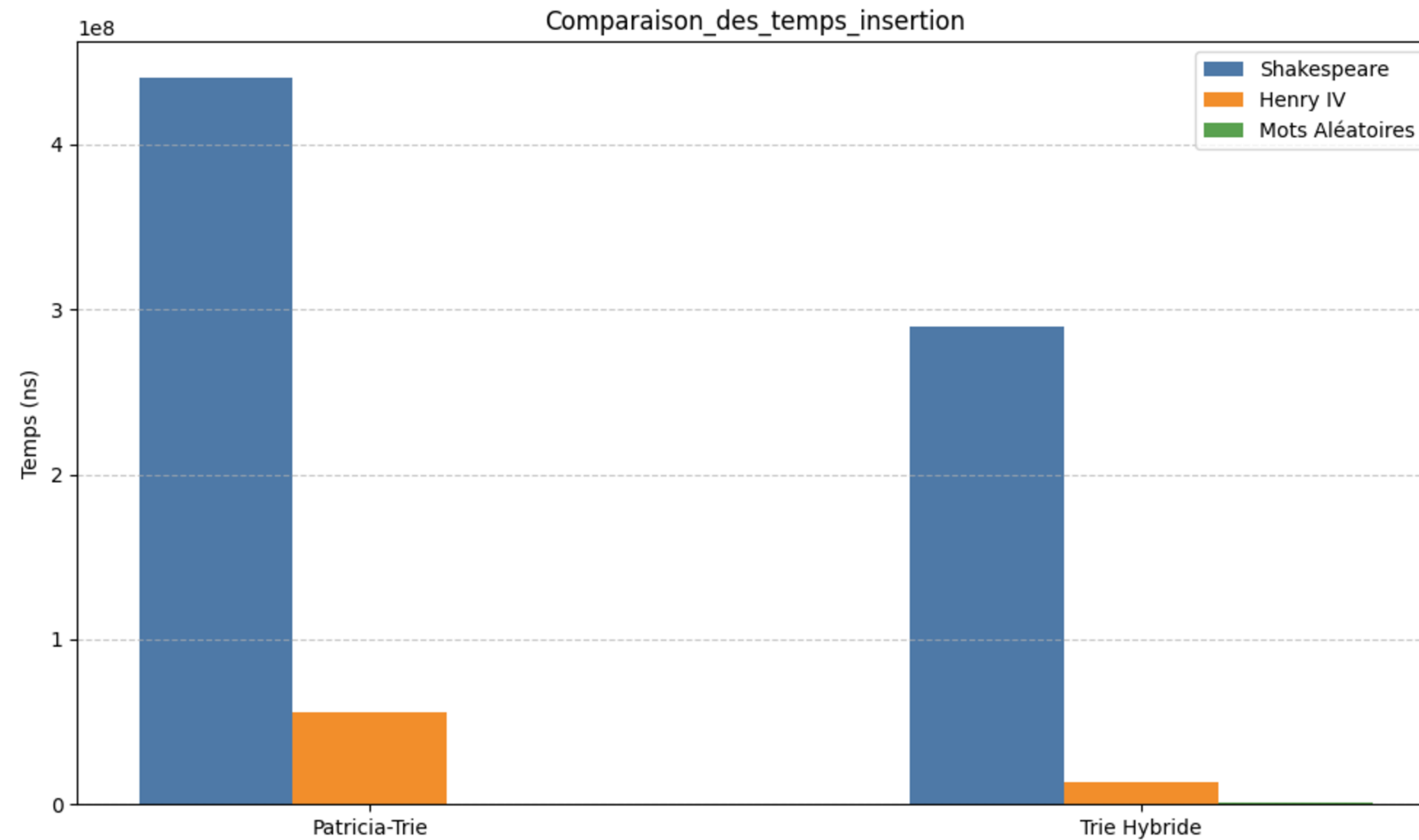
Étude expérimentale



Comparaison

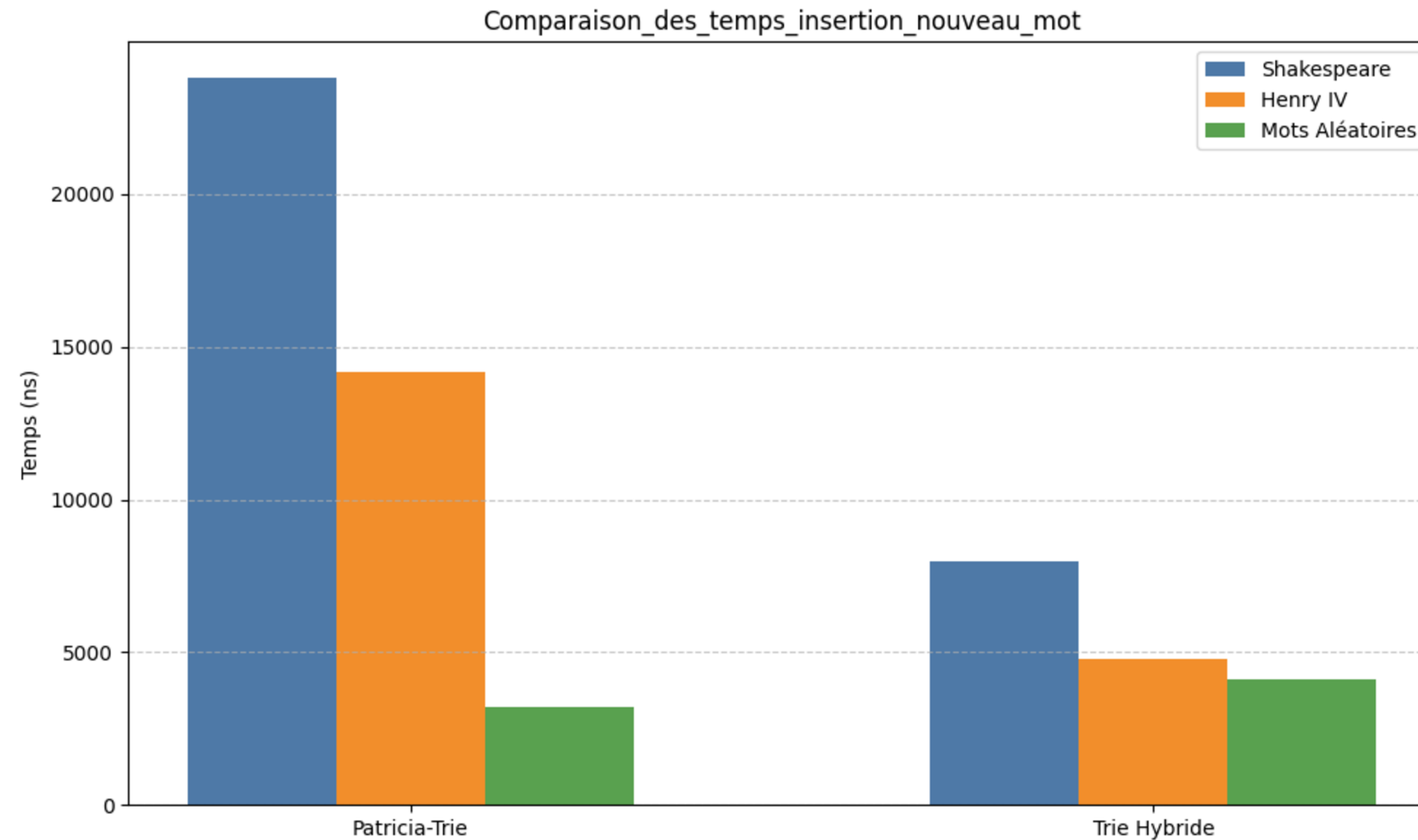
Critère	Patricia-Trie	Hybride Trie
Temps de construction	440265900 ns	289549100
Temps d'insertion d'un mot	23800 ns	8000 ns
Temps de suppression	20800 ns	12900 ns
Temps de recherche	1416200 ns	1139000 ns
Profondeur	6	16
Hauteur	11	36
Taille du fichier	5442035 octets	16868237 octets

Comparaison en terme de construction



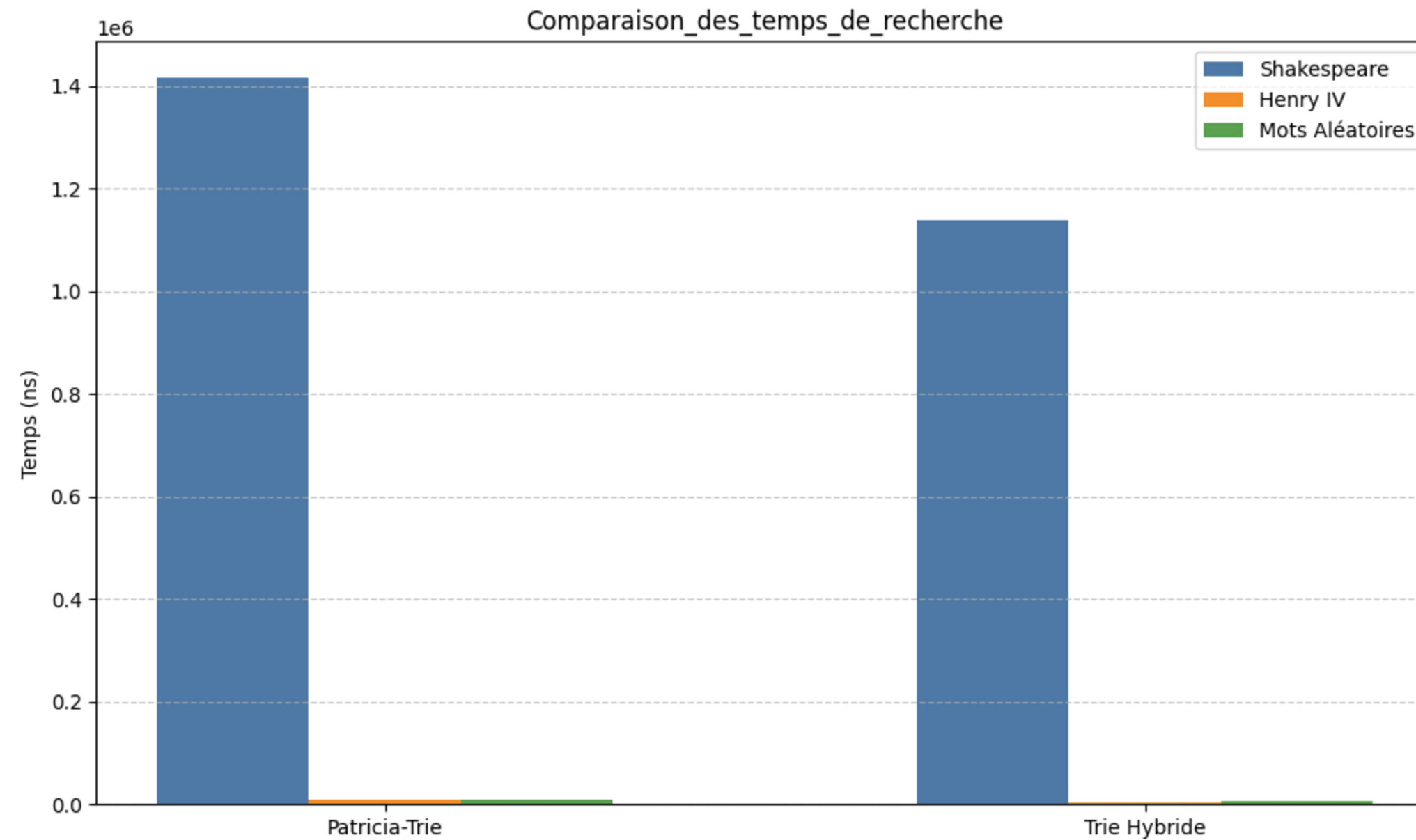
ce diagramme montre clairement que la construction se fait plus rapidement dans les tries hybrides, de ce fait le trie hybride est meilleur en terme de rapidité de construction.

Comparaison en terme d'insertion



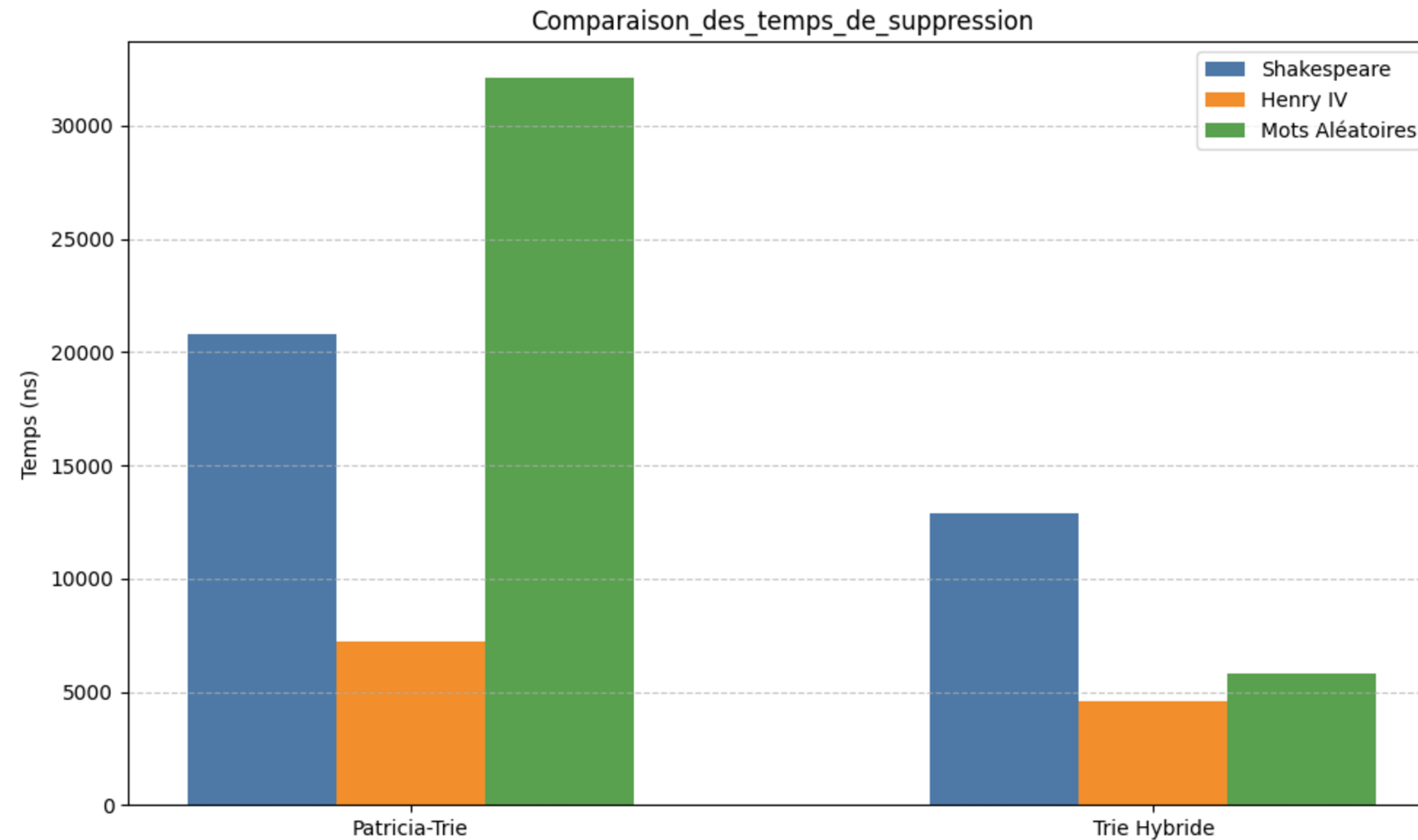
ce diagramme montre clairement que l'ajout d'un nouveau mot se fait plus rapidement dans les tries hibrydes, de ce fait le trie hibryde est meilleur en terme de rapidité.

Comparaison en terme de recherche



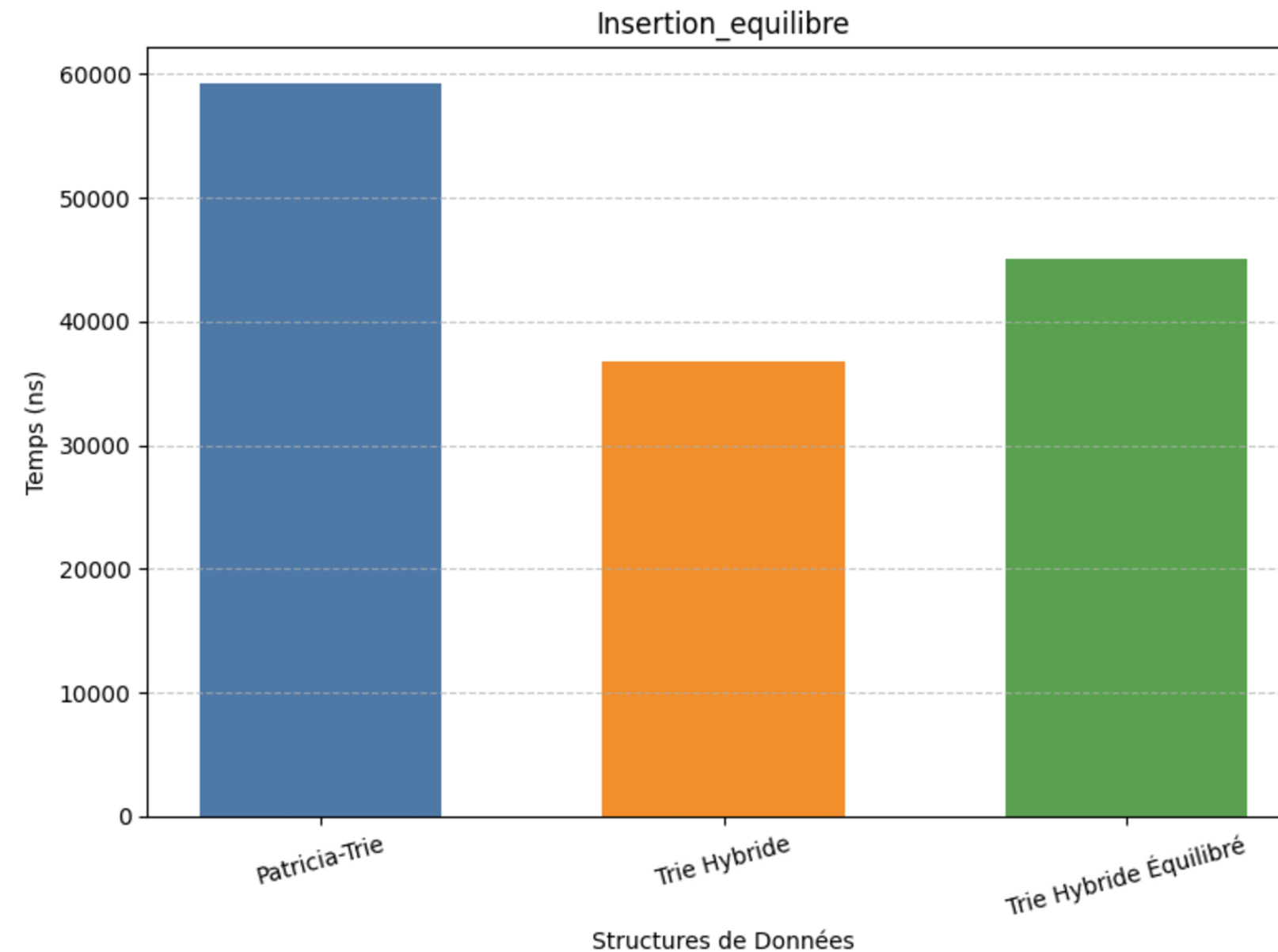
ce diagramme montre clairement que la recherche de mot se fait plus rapidement dans les tries hibrydes, de ce fait le trie hibryde est meilleur en terme de rapidité.

Comparaison en terme de suppression



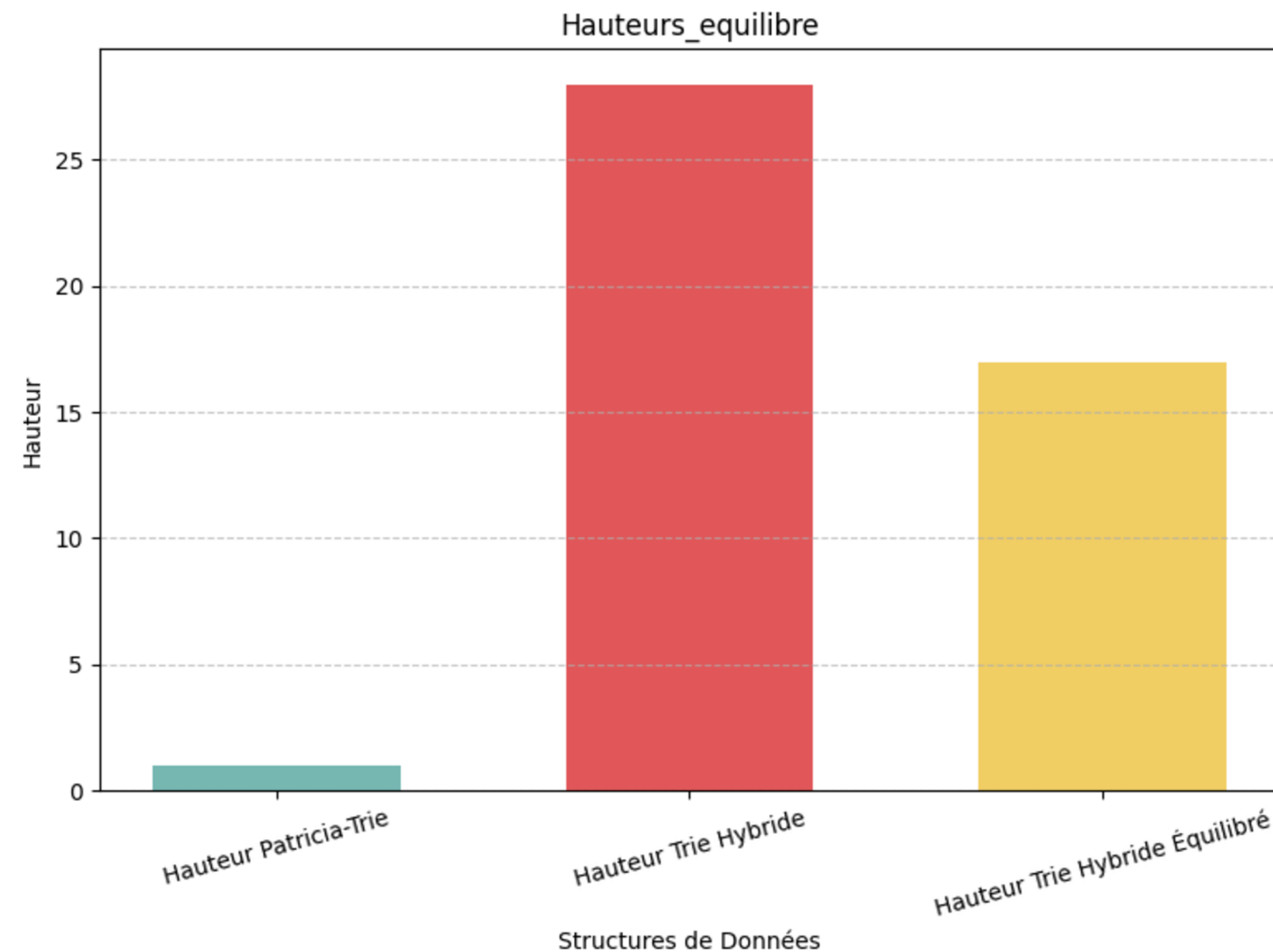
ce diagramme montre clairement que la suppression de mot se fait plus rapidement dans les tries hibrydes, de ce fait le trie hibryde est meilleur en terme de rapidité.

Comparaison en terme d'ajout



Ici on peut voir que le trie hybride est plus rapide que le trie hybride équilibré en terme d'insertion car le trie hybride équilibré doit faire beaucoup de comparaisons pour insérer un mot. Et la Patricia trie est toujours lente en terme d'insertion.

Comparaison en terme de hauteur



Dans ce diagramme, on peut voir que le trie hybride équilibré a une hauteur plus faible que le trie hybride non équilibré, mais on peut constater aussi que la hauteur de la patricia trie est plus faible que celle du trie hybride équilibré.

Hauteur profondeur

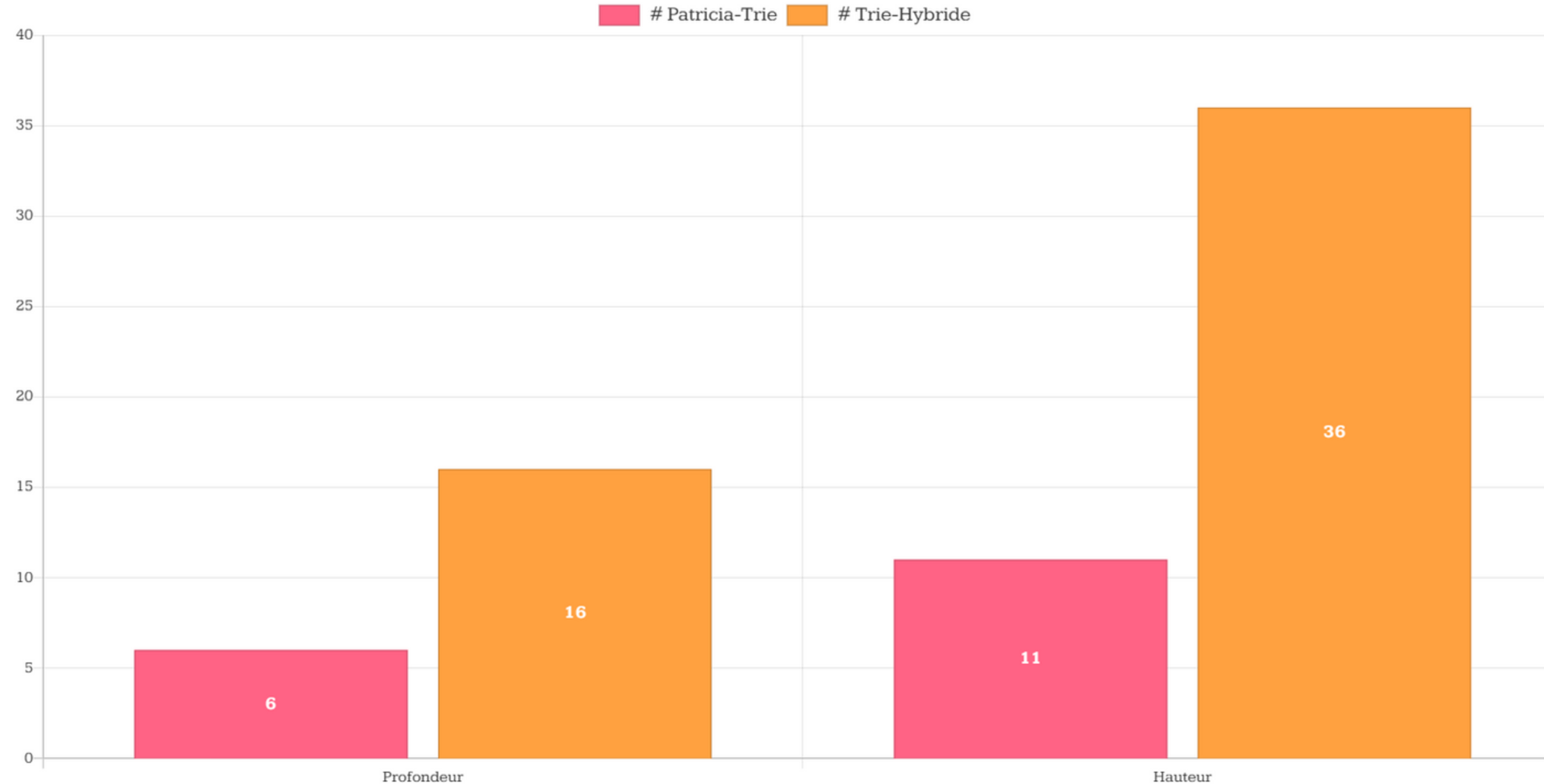


diagramme pour la profondeur et la hauteur des arbres

Taille fichier

pour la Patricia-Trie: 5442035 octets, pour le Trie Hybride: 16868237 octets.

d'après cette experimentation on a conclu que les patricia-tries est meilleur en terme d'occupation d'espace mémoire.

Conclusion

pour les deux type d'arbre qu'on a vu, chacune a ses avantages et inconvénients, la Patricia trie est meilleur en terme d'occupation d'espace mémoire, et moins bonne en terme de rapidité. pour la trie hibryde, elle comme avantage sa rapidité d'insertion en terme de temps et a comme inconvénient l'occupation de l'espace mémoire.



Thank You